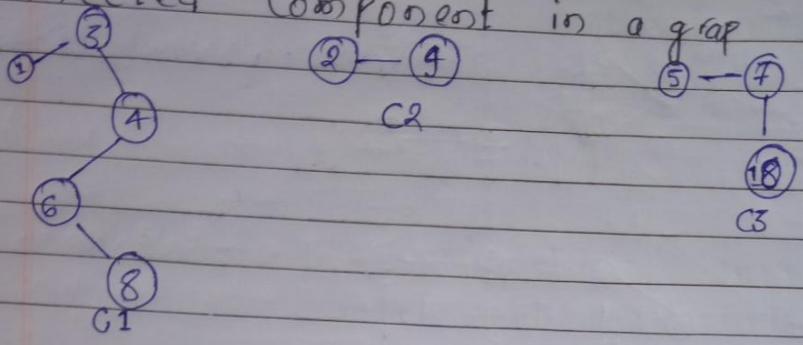
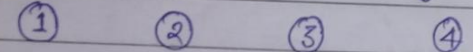
Connected Component in a graph



** Don't say above as 3 graph. rather say as a disconnected graph having 3 component



↓

called as graph have 4 Components

BFS / DFS

** whatever Code we write we have to write For multiple component

```
For(int i=1; i<=10; i++)
{
    IF(!vis[i])
    {
        dFS
        bFS   → technique
    }
}
```

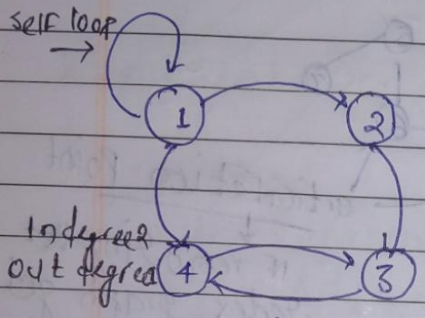
vis [0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0]
1 2 3 4 5 6 7 8 9 10

** assume every graph has disconnected Component & write your Code like this (Not assume graph having single Component)

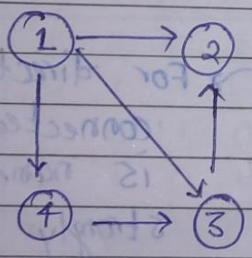
Graphs

Graph → Collection of vertices and edges

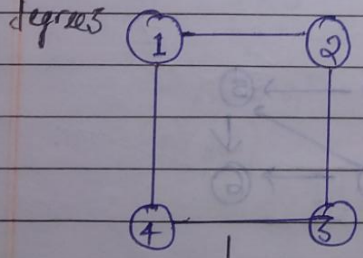
$G = (V, E)$ — V: set of vertices, E: set of edges



Directed graph → having direction



Simple Digraph → having no Parallel edges & self loop



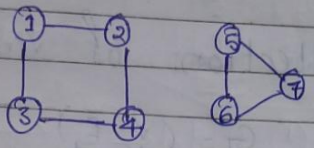
Graph / non-directed graph

Path:- a sequence of nodes or vertex such that none of the nodes are repeating or ~~visited twice~~

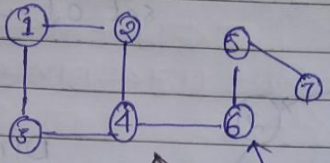
Total degree of all the nodes is twice of all the edges

Total = $2 \times E$

Not Connected



Connected

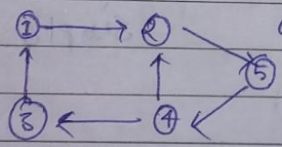


articulation point

↓

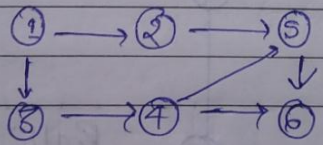
If removing that vertex graph get divide in multiple disconnected graph

strongly Connected →



↷ For directed graph connected graph is named as strongly connected

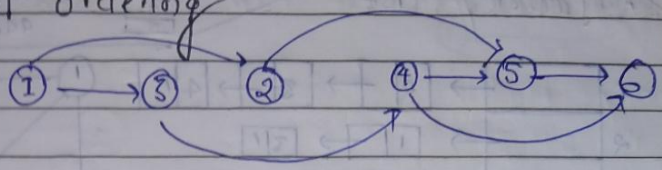
Directed acyclic graph



↓

by travelling in any direction you can't Form closed cycle.

Topological ordering



Representation of undirected Graph

Method
1. Adjacency Matrix
2. Adjacency List
3. Compact List

Adjacency Matrix

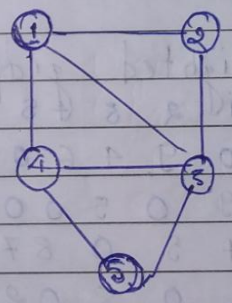
$A =$

	1	2	3	4	5
1	0	1	1	1	0
2	1	0	1	0	0
3	1	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

5 X 5

$n \times n = n^2$

$O(n^2)$



$G = (V, E)$

$|V| = n = 5$

$|E| = e = 7$

Represented as

N = 5

at every array int arr[6]

at every array we have pair — Pair<int, int> arr[6]

understand — vector<int> adj[6]

** 6 as vector

0 based indexing

at every array we have vector

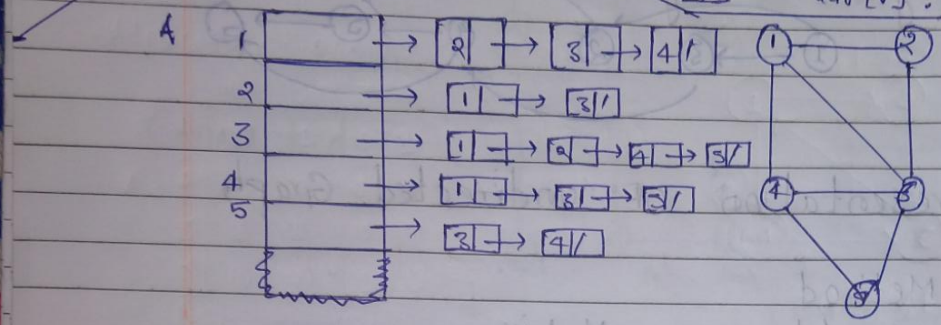
Reverse

Adjacency List

0	
1	(2, 3, 4)
2	(1, 3)
3	
4	
5	

→ O(n + 2E)

*** adj[u].push_back(v)
adj[v].push_back(u)

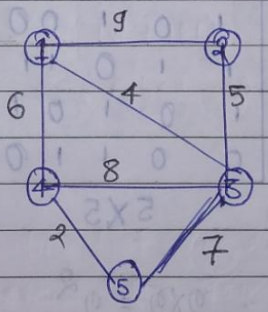


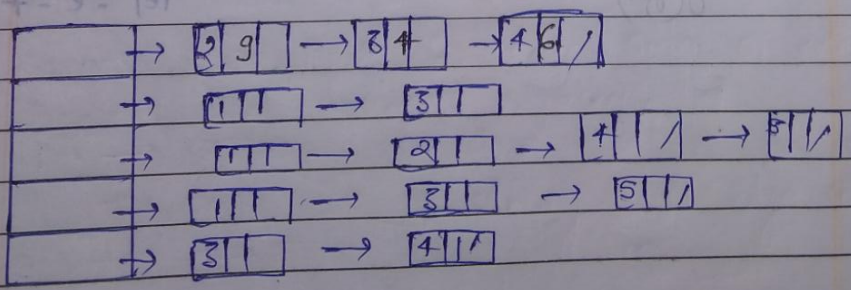
|V| + 2|E|

n + 2e ↝ we are going to every edge twice

Weighted graph

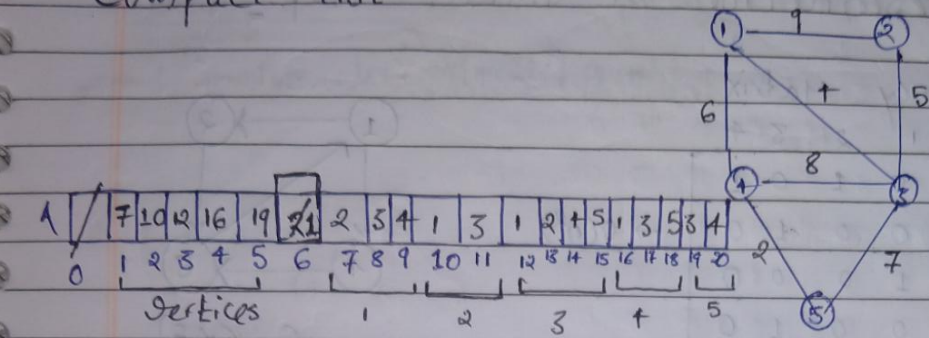
	1	2	3	4	5
1	0	9	4	6	0
2	9	0	5	0	0
3	4	5	0	8	7
4	6	0	8	0	2
5	0	0	7	2	0





Revise by Concentration

Compact list



G = (V, E)

* 7 & 10 are giving starting & ending vertices of index 1 → 7, 8, 9 — 10 stop (because at 2 10 is present)

|V| = n = 5

|E| = e = 7

|V| + 2|E| + 1 (For 21)

= 5 + 2×7 + 1 = 20

20 + 1 = 21 → not using 0

Order of space

↓

|V| + 2|E|

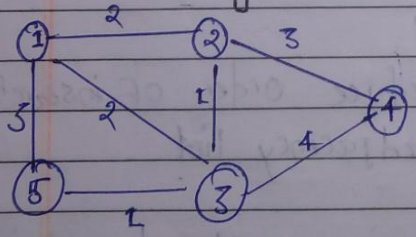
↓

n + 2e

n + 2n = 3n = O(n)

***Adjacency list For Problem solving

IF (weighted)



O(n + 2E) + 2E

vector < Pair < int, int > > adj[6];

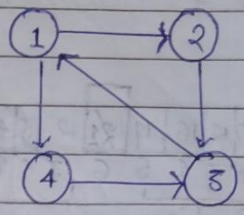
	adj node, wt
0	
1	((2,2),(3,2)(5,3))
2	
3	adj[y].push_back({v,wt});
4	
5	

Representation OF directed Graph

Adjacency Matrix

A =	1	2	3	4
1	0	1	0	1
2	0	0	1	0
3	1	0	0	0
4	0	0	1	0

→ $O(n^2)$



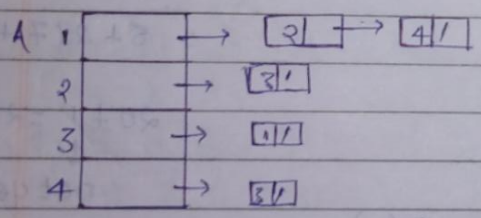
$G = (V, E)$

$|V| = 4$

$|E| = 5$

Indegree → See column
Outdegree → See row

Adjacency List



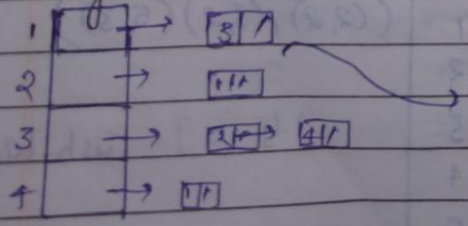
→ $O(|V| + |E|)$

$O(n+e) = O(n+n) = O(n)$

Outdegree → see row
*indegree → Traverse all the list (For ex:- For 3 see wherever 3 is found)

ex:- indegree of 3 is 2

suppose we have to reduce order of inserching indegree then inverse adjacency list



→ storing all edges coming in

Compact list

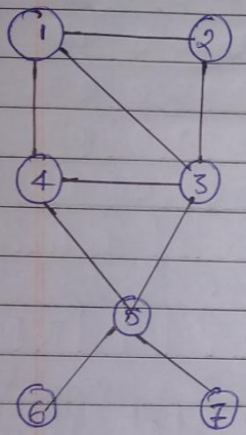
↓

similar like undirected graph

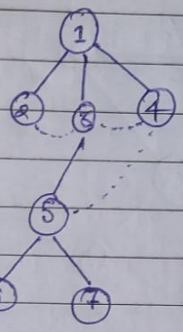
Breadth First Search

↓

similar to level order traversal of binary tree

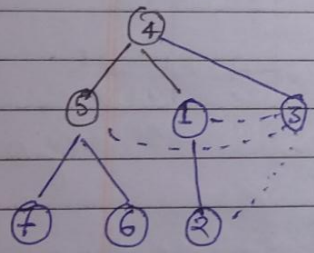


=



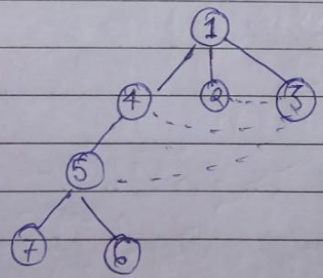
BFS:- 1, 2, 3, 4, 5, 6, 7

Or



BFS:- 4 5 1 3 7 6 2

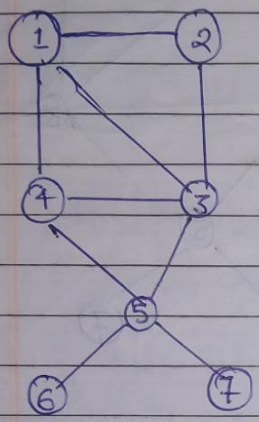
Or



BFS:- 1 4 2 3 5 7 6

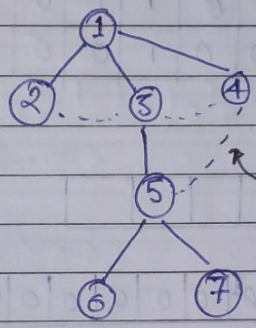
Rule:

1. Visiting
2. Exploring



BFS: 1 2 3 4 5 6 7

Queue:- ~~1~~ ~~2~~, ~~3~~, ~~4~~ ~~5~~ ~~6~~ ~~7~~.



→ O(v)

Analytical time

Cross edges

BFS spanning Tree

or

1. 1 4 3 2 5 ~~7~~ 6

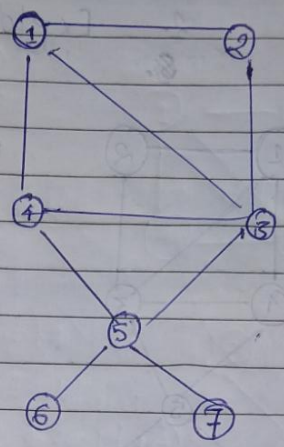
2. 5 7 3 6 4 2 1

3. 2 ~~3~~ ~~4~~ 5 1 6 7

* For one graph multiple breadth First search is possible

* The answer that we are getting is not important, visiting all the nodes are important

	0	1	2	3	4	5	6
0							
→1		0	1	1	1	0	0	0
2		1	0	1	0	0	0	0
3		1	1	0	1	1	0	0
4		1	0	1	0	1	0	0
5		0	0	1	1	0	1	1
6		0	0	1	0	1	0	0
7		0	0	0	0	1	0	0



q | 1 | | | | | | |

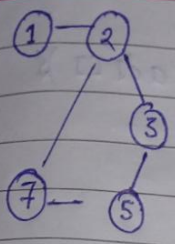
visited | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
0 1 2 3 4 5 6 7

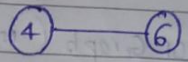
↷ starting vertex

```
void BFS (int i)
{   int v;
    printf("%d", i);
    visited[i] = 1
    enqueue(q, i)
    while(!q.empty())
    {
        v = dequeue(q);
        for(v=1; v<=n; v++)
        {   if(A[v][v]==1 && visited[v]==
            {
                printf("%d", v);
                visitited[v]=1;
                enqueue(q,v);
            }
        }
    }
```

O(n²) ←

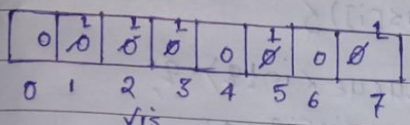
*BFS → Traversing adjacent node First





1	2
2	1 3 7
3	2 5
4	6
5	3 7
6	4
7	2 5

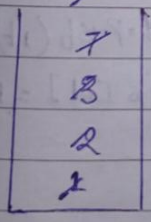
1 2 3 7 5 4 6




```
For ( i = 1 → n)
{
    if ( vis[i] == 0)
        bFS(i)
}
```

→ as a graph can have multiple disconnected component
*** So always write For loop

BFS (1)

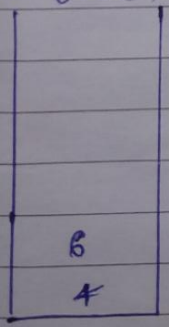


whenever you take out node print in BFS

node = 1
node = 2
node = 3
node = 5

TC → O(n)
SC → O(N) ~ For visited array space

BFS (4)



node = 4
node = 6

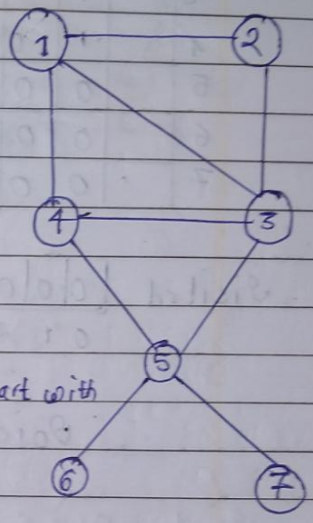
```
vector<int> bfsOfGraph(int V, vector<int> adj[]){
    vector<int> bfs;
    vector<int> vis(V+1,0);
    for(int i=1; i<=V; i++){
        if(!vis[i]){
            queue<int> q;
            vis[i]=1;
            while(!q.empty()){
                int node = q.front();
                q.pop();
                bfs.push_back(node);
                for(auto it: adj[node]){
                    if(!vis[it]){
                        q.push(it);
                        vis[it]=1;
                    }
                }
            }
        }
    }
    return bfs;
}
```

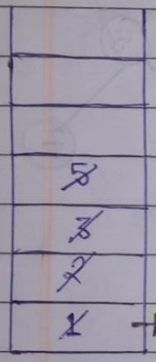
you can also make separate BFS fn

Depth First Search

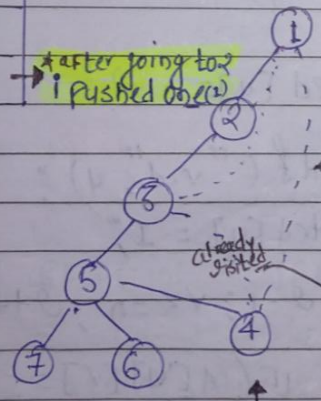
* Similar to Preorder Traversal of Binary Tree

1. Visited
2. Exploring





* you can start with any vertex



← Back edge

O(n)
↓
analytical time

* after coming to 4, we are seeing.

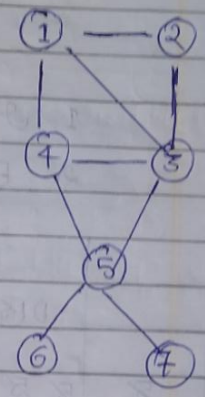
Depth First search spanning Tree

Or

1. 1, 3, 5, 4, 2, 7, 6, 2
2. 1, 2, 3, 4, 5, 6, 7
3. 1, 4, 5, 7, 3, 2, 6,
4. 4, 1, 3, 5, 6, 7, 2

→ * Various valid Depth First search

	0	1	2	3	4	5	6	7
0								
1		0	1	1	1	0	0	0
2		1	0	1	0	0	0	0
3		1	1	0	1	1	0	0
4		1	0	1	0	1	0	0
5		0	0	1	1	0	1	1
6		0	0	0	0	1	0	0
7		0	0	0	0	1	0	0



visited

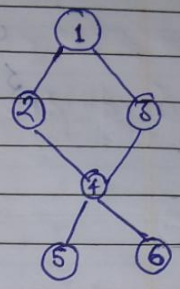
0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

```
void DFS (int u)
{
    if (visited[u] == 0)
    {
        printf ("%d", u);
        visited[u] = 1;
        for(v = 1; v <= n; v++)
        {
            if (A[u][v] == 1 && visited[v] == 0)
                DFS(v);
        }
    }
}
```

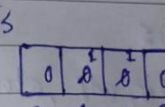
To avoid calling already visited

→

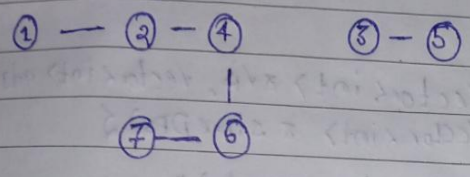
```
#include <stdio.h>

int main()
{
int G[7][7] = {{0,0,0,0,0,0,0},
               {0,0,1,1,0,0,0},
               {0,1,0,0,1,0,0},
               {0,1,0,0,1,0,0},
               {0,0,1,1,0,1,1},
               {0,0,0,0,1,0,0},
               {0,0,0,0,1,0,0}};

  BFS(G, 1, 7);

}

void BFS(int G[][7], int start, int n)
{       int i = start
      struct Queue q;
       int visited[7] = {0};

    Printf("%d", i);
       visited[i] = 1;
        enqueue(i);
```



```
while( !isEmpty() )
{       i = dequeue();

        For( j=1 ; j<n ; j++)
        {
            IF ( G[i][j]==1 && visited[j]==0)
            {
                printF("%d", j);
                visited[j] = 1;
                enqueue(j);
            }
        }
}

void DFS( int G[][7], int start, int n)
{
    static int visited[7] = {0};
    int j;
    IF ( visited[start] == 0)
    {
        printF("%d" start);
        visited[start] = 1;
        For( j=1, j<n, j++)
        {
            IF ( G[start][j]==1 && visited[j]==0)
            DFS(G, j, n);
        }
    }
}
```



DFS → you have to go depth First



adj list

1 → 2

2 → 1 4 7

3 → 5

4 → 2 6

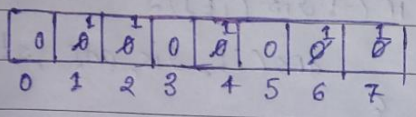
5 → 3

6 → 7 4

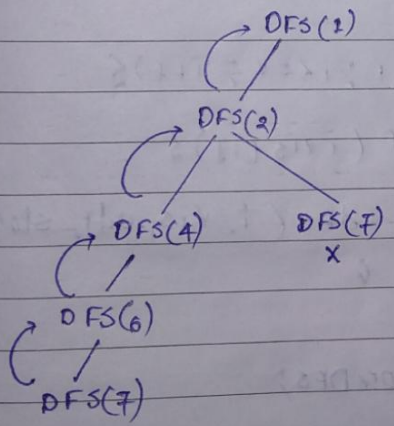
7 → 2 6

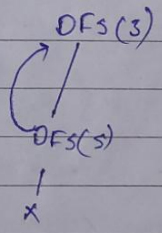
1 2 4 6 7 3 5

vis




```
For ( i = 1 → 7)
{
    IF (! vis[i])
        dfs(i)
}
```





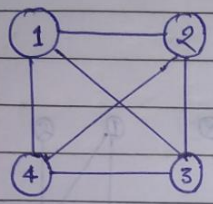
TC → O(N) + O(N)

SC → O(N)

```
void dfs(int node, vector<int> &vis, vector<int> adj[],
                    vector<int> &storeDFS) {
    storeDFS.push_back(node);
    vis[node] = 1;
    for(auto it: adj[node]) {
        if(!vis[it]) {
            dfs(it, vis, adj, storeDFS);
        }
    }
}

vector<int> dfsOfGraph(int V, vector<int> adj[]) {
    vector<int> storeDFS;
    vector<int> vis(V+1, 0);
    for(int i=1; i<=V; i++) {
        if(!vis[i]) {
            dfs(i, vis, adj, storeDFS);
        }
    }
    return storeDFS;
}
```

MST → Minimum cost spanning Tree

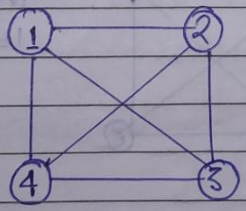
Spanning Tree



1. Spanning Tree
2. Prims MST
3. KrusKals MST

1. Spanning Tree

→ It is a subgraph of a graph having all vertices of graph, but n-1 edges so that there is no cycle Formed by those edges and graph must be connected

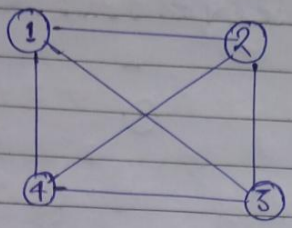


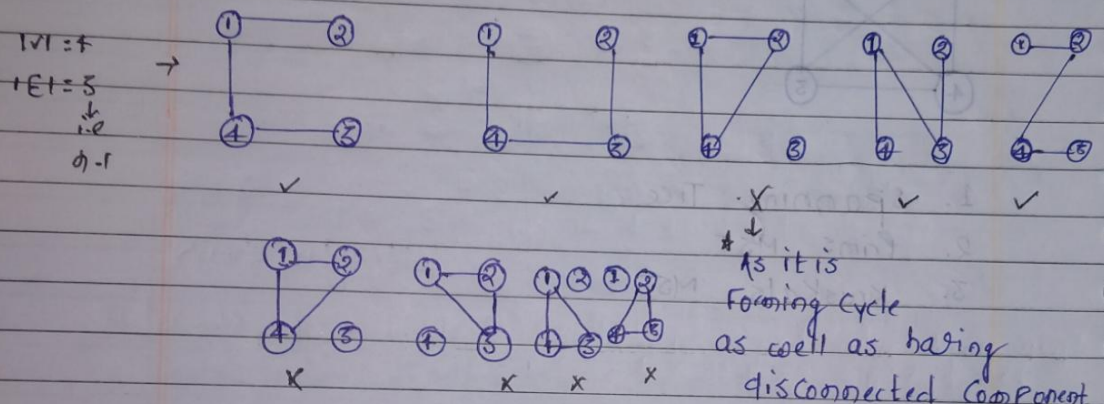
$G = (V, E)$

$n = |V|$

$e = |E|$

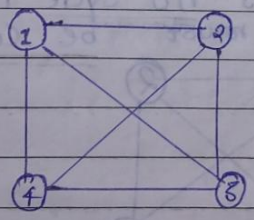
★ Mathematic def^n	$S \subseteq G$ $S = (V', E')$ → $\|V'\| = \|V\|$ $\|E'\| = \|V\| - 1$





* As it is Forming Cycle as well as having disconnected Component

No. OF possibilities OF spanning Trees



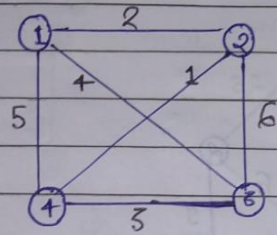
|V| = 4
|E| = 6

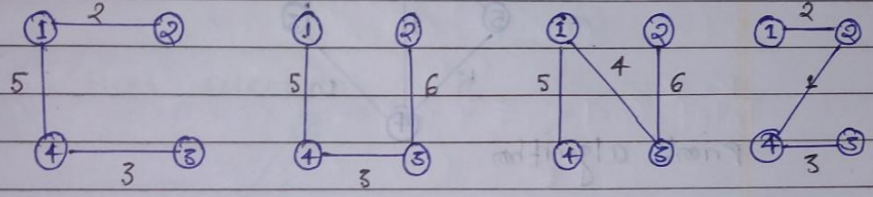
$$\binom{|E|}{|V|-1} - \text{Cycles}$$

↳ i.e 4-1

$$\binom{6}{3} - 4 = 20 - 4 = 16$$

Minimum cost spanning Tree





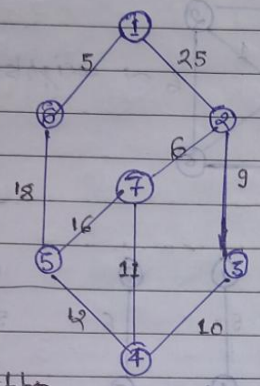
Total cost → 10 14 15 6

* Out of 16 possible case we have to Find spanning our aim is to Find minimum cost spanning tree

↓

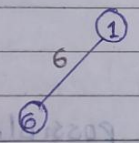
For it there are various algorithms

Prim's Minimum Cost spanning Tree

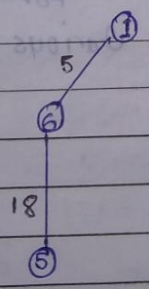


Prim's algorithm

step 1:- Select Minimum cost edge from graph

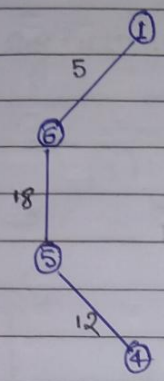


step 2:- Select Next minimum cost edge and/but it must be connected to already selected vertices

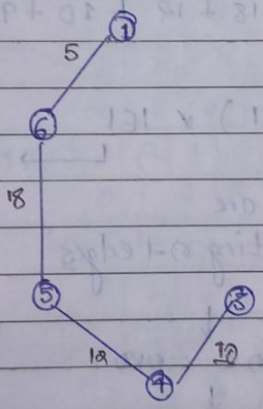


step 3:- Repeat the same thing

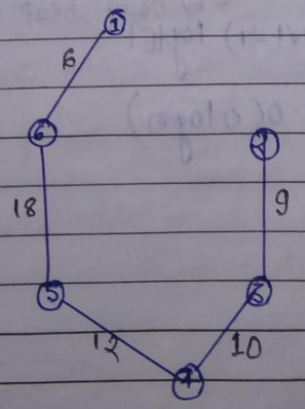
Next selection



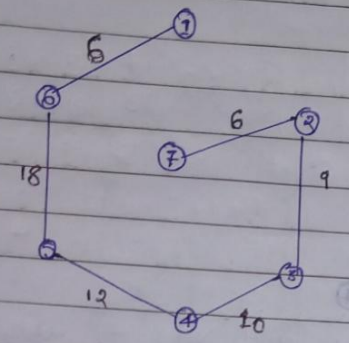
Next selection



Next selection



Next selection



Total Cost → 5 + 18 + 12 + 10 + 9 + 6 = 60

Order → $(|V| - 1) \times |E|$

- $(|V| - 1)$ → we are selecting n-1 edges
- $|E|$ → From every time we are finding minimum one from connected edges
  - let's we are exploring all edges
  - From there we are finding connected edges

$n \cdot e = n \times n$

$O(n^2)$

can be reduced to by using heap DS

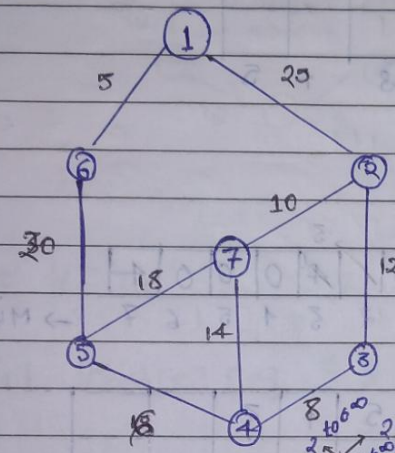
$(|V| - 1) \log |E|$

$= O(n \log n)$

* observe it, it is symmetric about diagonal. For Find Min^{m} value just traverse any one of lower triangular & upper triangular

- → means ∞ ↓ MAX-INT

Prim's program



cost

	0	1	2	3	4	5	6	7
0	-	-	-	-	-	-	-	-
1	-	-	25	-	-	-	5	-
2	-	25	-	12	-	-	-	10
3	-	-	12	-	8	-	-	-
4	-	-	-	8	8	16	-	14
5	-	-	-	-	16	-	20	18
6	-	5	-	-	-	20	-	-
7	-	-	10	-	14	16	-	-

2 to 1 25 → so 25 is less work as 1 & 2 shows 2 is nearest to 1

2 to 3 ∞ → work anyone (as equal)

0 means we have included it ←

near

	0	1	2	3	4	5	6	7
near	-	0	1	6	6	6	0	6

* updated below after Finding/Filling next one

* After Filling 000 Find Min^{m} connected then Fill next one of below i.e 5,6

t

	0	1
0	1	5
1	6	6

→ To store spanning Tree (vertex & edge)

* this is next edges Found after Finding Min^{m} From near array

→ Now Repeat above steps

near table of 5 & 6

	0	1	2	3	4	5	6	7
near	-	0	1	6	6	0	0	6

K = 5

checking for 5, is it nearer to 5 than 2

2 to 5 ∞ So 1 is better

→ Find Min^{m} & then Fill both (4,5)

near

-	0	1	6	5	0	0	5
0	1	2	3	4	5	6	7

→ Min = one is (3,4)

t

	0	1	2	3	4	5
0	1	5	4			
1	6	6	5			

next →

near

-	0	1	4	0	0	0	4
0	1	2	3	4	5	6	7

K = 3 → Min (2,3)

t

	0	1	2	3	4	5
0	1	5	4	3		
1	6	6	5	4		

next

near

-	0	0	0	0	0	0	4
0	1	2	3	4	5	6	7

t

	0	1	2	3	4	5
0	1	5	4	3	2	
1	6	6	5	4	3	

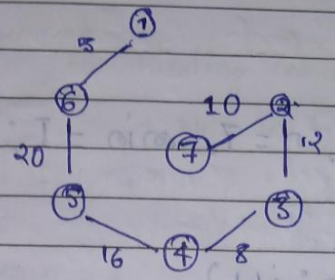
next →

near

-	0	0	0	0	0	0	2
0	1	2	3	4	5	6	7

t

	0	1	2	3	4	5
0	1	5	4	3	2	2
1	6	6	5	4	3	7



→

define I 32767 (∞)

int cost[8][8] = {{I, I, I, I, I, I, I, I},
{I, I, 25, I, I, I, 5, I},
{I, 25, I, 12, I, I, I, 10},
⋮
};

int near[8] = {I, I, I, I, I, I, I, I};

int t[2][6];

void main()
{
⋮

```
void main()
{
    int i, j, k, u, v, n = 7, min = I;

    For (i=1; i<=n; i++)
    {
        For (j=i; j<=n; j++)
        {
            IF (cost[i][i] < min)
            {
                min = cost[i][i];
                u=i, v=j;
            }
        }
    }

    t[0][0] = u; t[1][0] = v;
    near[u] = near[v] = 0;

    For (i=1; i<=n; i++)
    {
        IF(near[i] != 0)
        IF( cost[i][u] < cost[i][v])
            near[i] = u;
        else
            near[i] = v;
    }
}
```

* Finding min of vertices/edges from upper triangle matrices

near

-	0	-	-	-	-	0	-
0	1	2	3	4	5	6	7

t

	0	1
0	1	
1	6	

→ Now repetitive steps starts

↓

For finding remaining edges

↳ Now in t index is empty From 1

```
For (i=1; i<n-1; i++)        ↳ till 5
{
    min = ∞;
    For (j=1; j<=n; j++)
    {
        IF (near[j]!=0 && cost[j][near[j]] < min)
        {
            min = cost[j][near[j]];
            k = j;
        }
    }
```

Finding min From near array ←

-	0	1	6	6	0	0	6
0	1	2	3	4	5	6	7

```
    t[0][i] = k;  t[1][i] = near[k];
    near[k] = 0;
    For (j=1; j<=n; j++)
    {
        IF (near[j]!=0 && cost[j][k] < cost[j][near[j]])
            near[j] = k;
    }
}
Print t;
```

Updating/Filling t

Updating near array ←

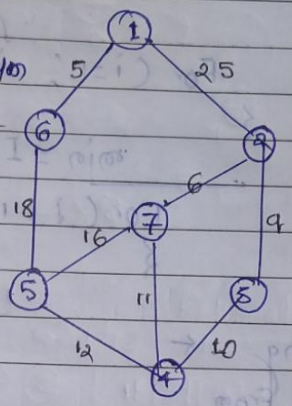
t

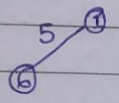
0	1	5
1	6	6

n=7

Kruskal Minimum cost spanning Tree

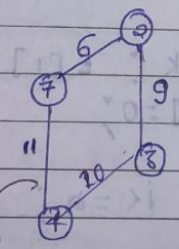
Rule :- Always select Minimum cost edge, but make sure they are not Forming a cycle
we have to select Min edge every time



$(|V|-1)|E|$ ← 

selecting edges

Find min cost edges From all edges



Forming a cycle, we will not select this edge (skip it)

$e = n \times n$

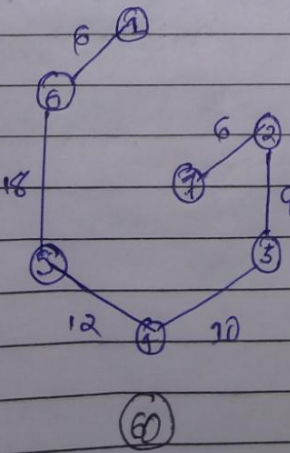
$O(n^2)$

↓

We can reduce complexity using heap

↓

$O(n \log n)$

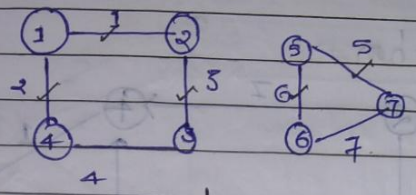


← After Final selection

(60)

ex:-

disconnected graph



↓

IF we apply Kruskal it will select only marked one

↓

*it is Finding spanning tree OF individual component but not whole graph

↓

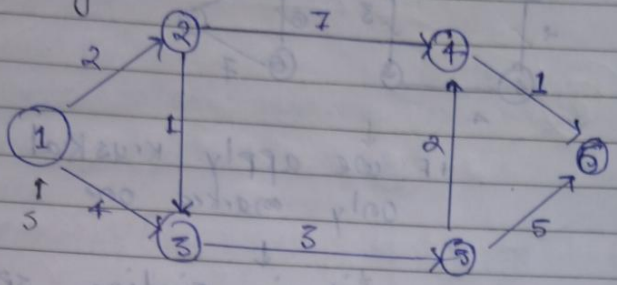
it is not possible in Prim's algorithm

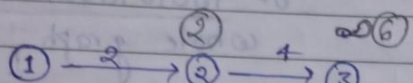
Difference b/w approaches OF prim's and kruskal

Prims:- Focus is more in Forming Tree.

kruskal:- Focus is more in Finding minimum

Dijkstra Algorithm

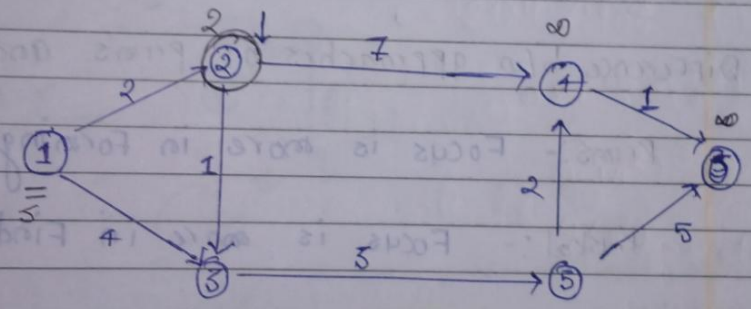




Relaxation

IF ($d[u] + c(u,v) < d[v]$)
cost

$d[v] = d[u] + c(u,v)$

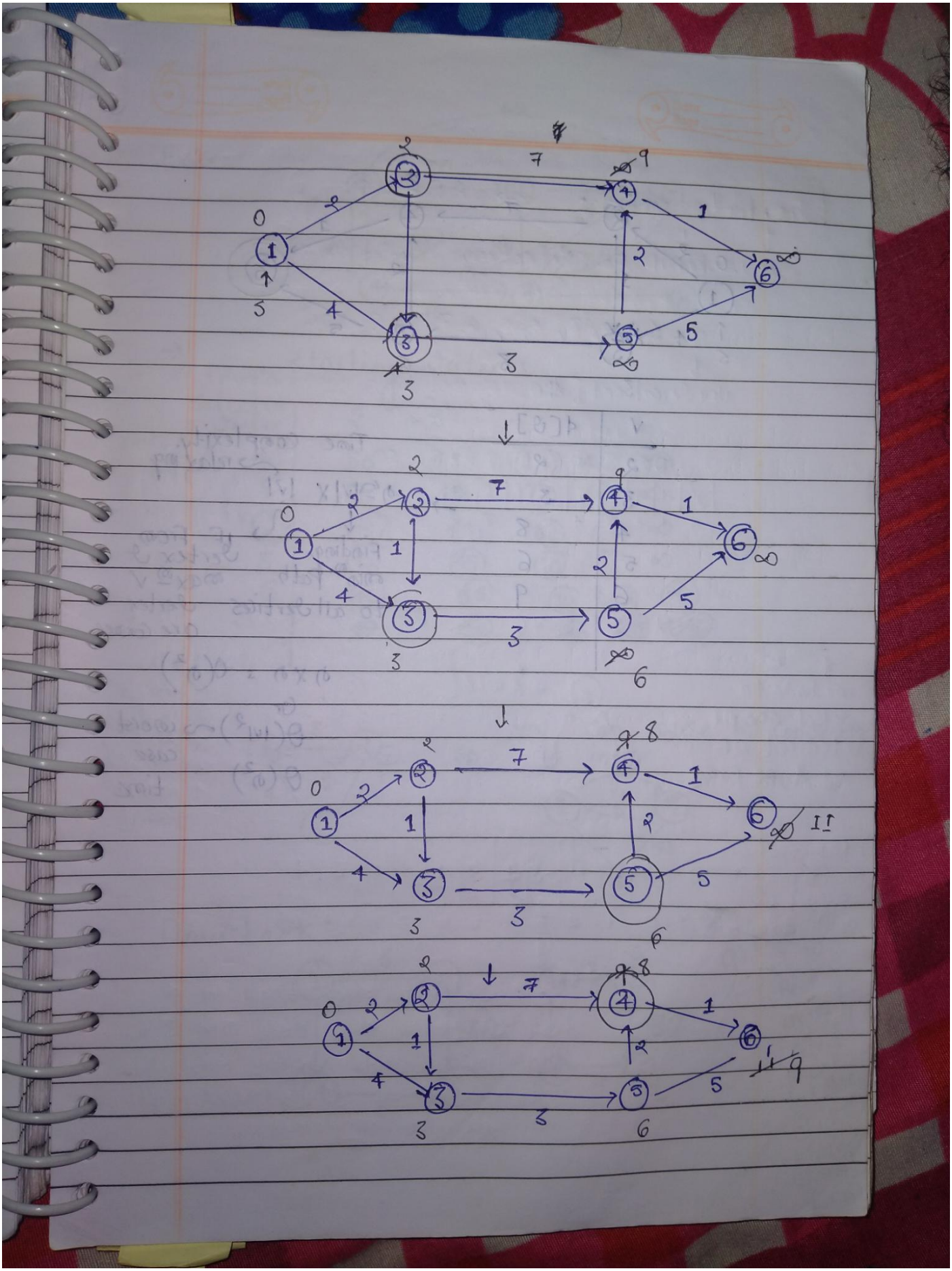


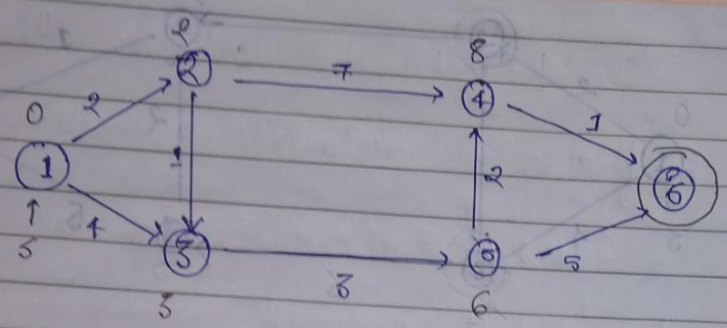
Now select min(2) & perform relaxation

* IF no direct Path possible then place ∞

↓

Now repeating step starts





V	d[v]
2	2
3	3
4	8
5	6
6	9

Time complexity. → relaxing

$n = |V| \times |V|$

↓ Finding min path to all vertices

↳ If From vertex 1 max V vertex are connected

$n \times n = O(n^2)$

or

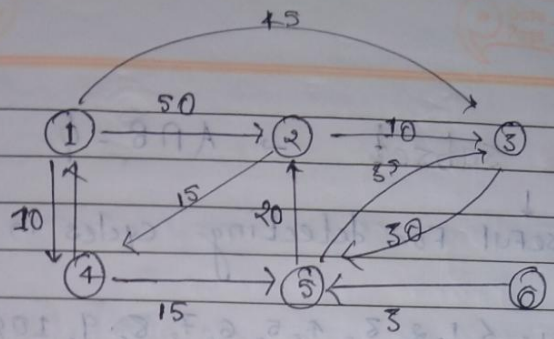
$\Theta(|V|^2)$ ~ worst case time

$\Theta(n^2)$

Ex:-

Ex:-

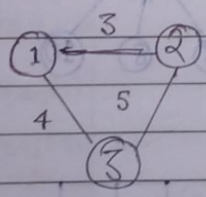
Ex:-



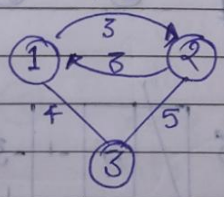
→ starting vertex 1

selected vertex	2	3	4	5	6
4	50	45	10	∞	∞
5	50	45	10	25	∞
2	45	45	10	25	∞
3	45	45	10	25	∞
6	45	45	10	25	∞

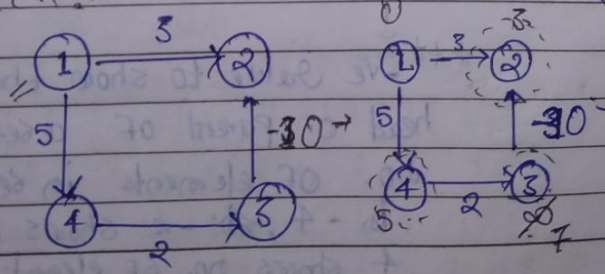
Ex:-



So, you can solve by converting it into



Drawback:-



Dijkstra say not to check already selected one but here check ② should be updated by 3.30 here fast

Disjoint subset → $A \cap B = \phi$

↓

useful for detecting cycles in graph

$U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

↓

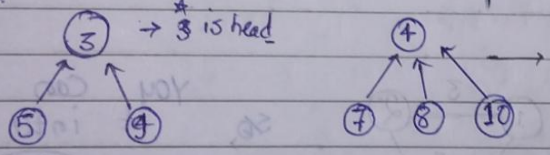
universal set [Also assumed that every element is set in itself]

	①	②	③	④	⑤	⑥	⑦	⑧	⑨	10
	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
0	1	2	3	4	5	6	7	8	9	10

* -ve No. shows it is set or head of set

$A = \{3, 5, 7\}$ $B = \{1, 7, 8, 10\}$ $A \cap B = \phi$

* we have to select one of element of set A as parent



* arrow represents ⑦ is element of that set in which 4 is also present

Representation of Disjoint subset in an universal set

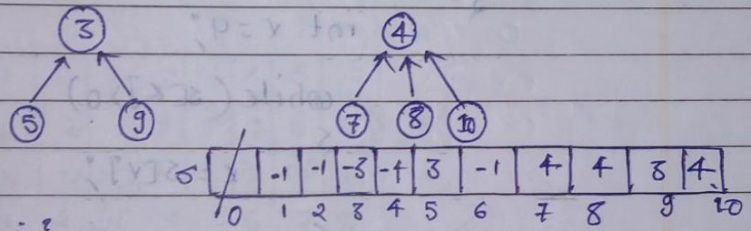
	-1	-1	-3	4	3	-1	4	4	5	4
0	1	2	3	4	5	6	7	8	9	10

*** -ve value to show that it is head or parent of a set and 3 for no. of elements in set, similarly in -4, -ve shows parency and 4 shows no. of element

- Union
- Find

Union | Weighted union

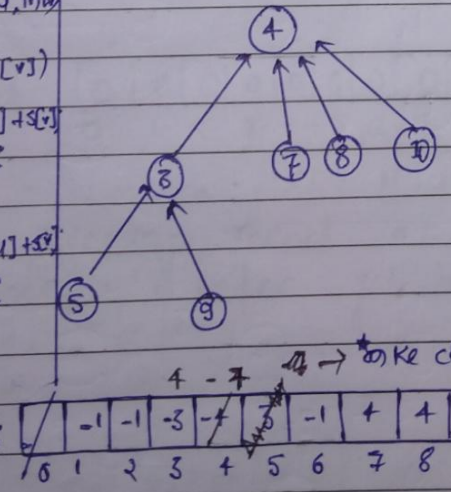
A = {3, 5, 9} B = {4, 7, 8, 10}



A∪B = ?

A ∪ B = {3, 4, 5, 7, 8, 9, 10}

```
void union(int u, int v)
{
  IF(S[u]<S[v])
  { S[u] = S[u]+S[v];
    S[v]=u;
  }
  else
  { S[v] = S[u]+S[v];
    S[u]=v;
  }
}
```



* In union, the set having more NO. OF element make parent of that that's why called (weighted union)

* In A ∪ B who is parent of set A → 3 & Parent of set B → 4 & in -3 & -4 which is smaller make parent of 3 as 4 and at place of -7 replace by -4

Note:- For Finding / checking loop, let's check Parent of 5 is 4 & Parent of 10 is 4 so they belong to same set so there is cycle

Find

↓

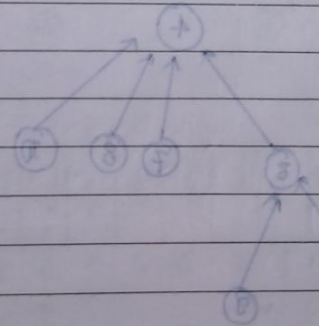
it is Finding OF root node OF any element

ex:- Parent OF 5 is 4

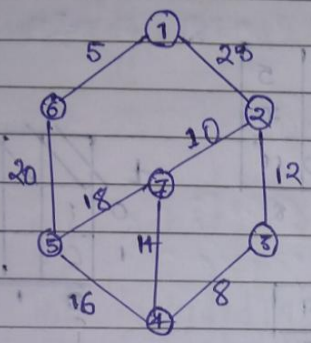
Parent OF 9 is 4

↷ x:- 9

```
int Find (int u)
{
    int x=u;
    while(s[x]>0)
    {
        x=s[x];
    }
    return x;
}
```



Kruskal's program (very simple)



t

	0	1	2	3	4	5
0						
1						

edges

	0	1	2	3	4	5	6	7	8
0	1	1	2	2	3	4	4	5	5
1	2	6	3	7	4	5	7	6	7
2	25	5	12	10	8	16	14	20	18

Set

X	-1	-1	-1	-1	-1	-1	-1
0	1	2	3	4	5	6	7

→ * Set data structure is used to check looping cycle or not

included

0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8

↓

For keeping record of which edge index get included

Tracing :- There is no initial step, every step is repeating step.

edges

	0	1	2	3	4	5	6	7	8
0	1	1	2	2	3	4	4	5	5
1	2	6	3	7	4	5	7	6	7
2	25	5	12	10	8	16	14	20	16

j → smallest

t

	0	1	2	3	4	5
0	1					
1	6					

set

0	1	2	3	4	5	6	7
X	~~6~~	-1	1	-1	-1	~~2~~	-1

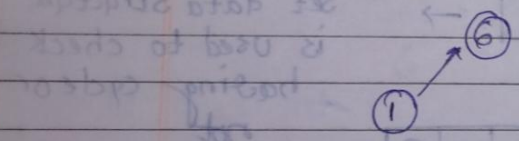
→ * After searching make union of that

includede

0	1	2	3	4	5	6	7	8
0	1	0	0	0	0	0	0	0

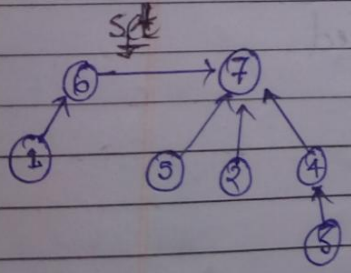
↓

For keeping record of which edge index got included



Now by doing repeatedly

Note :- Trace by self (using Pen & Paper)



t

	0	1	2	3	4	5
0	1	3	2	2	4	5
1	6	4	7	3	5	6

set

0	1	2	3	4	5	6	7
X	6	7	4	7	7	-2	-5

included

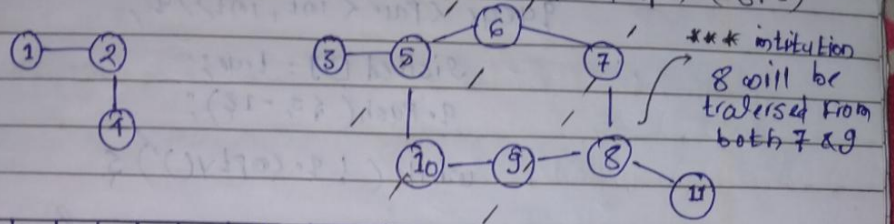
0	1	2	3	4	5	6	7	8
0	1	1	1	1	1	1	1	1

```
#define I 32767

int edges[3][4] = { {1,1,2,2,3,4,4,5,5},
                    {2,6,3,7,4,5,7,6,7},
                    {25,5,12,10,18,16,14,20,18} };

int set[8] = {-1,-1,-1,-1,-1,-1,-1,-1,-1};

int included[9] = {0}, t[2][6];

void union(int u, int v)
{
        IF (set[u] < set[v])
        {
            set[u] = set[u] + set[v];
            set[v] = u;
        }
        else
        {
            set[v] += set[u];
            set[u] = v;
        }
}
```

```
void main()
{
    int i=0, j, k, n=7, e=9, min=I, u, v;
    while (i<n-1)
    {
        min=I;
        For(j=0; j<e; j++)
        {
            if(included[j]==0 && edges[2][j]<min)
            {
                min = edges[2][j];
                k=j; u=edges[0][j];
                v=edges[1][j];
            }
        }
        if (Find(u) != Find(v))
        {
            t[0][i]=u; t[1][i]=v;
            Union(Find(u), Find(v));
            i++;
        }
        included[k]=1;
    }

    For(i=0; i<n-1; i++)
        cout << "(" << t[0][i] << ", " << t[1][i]
             << ")" << endl;
}
```

For Finding out Min

Q.
Cycle Detection in Undirected Graph (BFS)



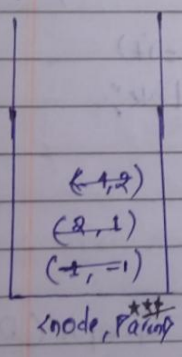
	0	1	2	3	4	5	6	7	8	9	10	11
Vis	0	0	0	0	0	0	0	0	0	0	0	0

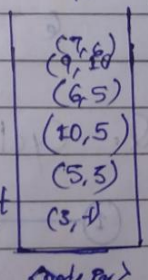
Adj list	
1	2
2	1 4
3	5
4	2
5	3 10 16
6	5 7
7	6 8
8	7 9 11
9	10 8
10	5 9
11	8

```
For ( i = 1 - N )
{
    IF ( ! Vis[i] )
        IF (CycleBFS(i))
            return True;
}
```

CycleBFS(i)

~~node = 1 Par = -1~~
~~node = 2 Prev = 1~~
node = 4 Prev = 2
node = 3 Prev = -1
node = 5, Prev = 3





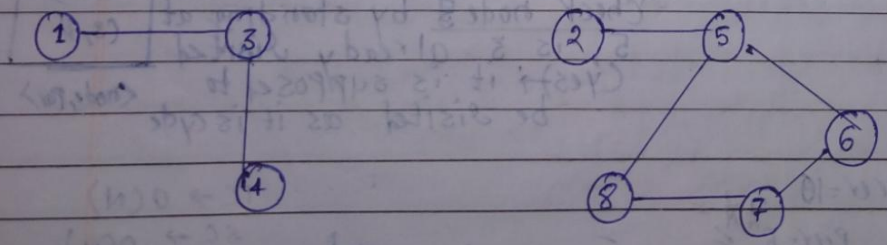
★ check node 3 by standing at 5, is 3 already visited (yes) it is supposed to be visited as it is cycle

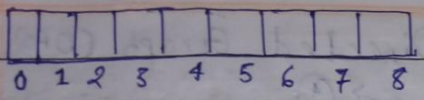
node:9 Prev=10
→ node = 7, Prev = 6 **
standing at 7
check adjacent node 6, it is supposed to be visited (it is parent)
" " " 8, it is not supposed to be visited
return true

TC → O(N)
SC → O(N)

```
bool cycledetect(int s, int V, vector<int> adj[], vector<int> &visited)
{
    queue<pair<int,int>> q;
    visited[i] = true;
    q.push({s, -1});
    while(!q.empty()){
        int node = q.front().first;
        int par = q.front().second;
        q.pop();
        for(auto it : adj[node]){
            if(!visited[it]){
                visited[it] = true;
                q.push({it, node});
            }
            else if(par != it)
                return true;
        }
    }
    return false;
}
```

Q. Cycle Detection in Undirected Graph (DFS)

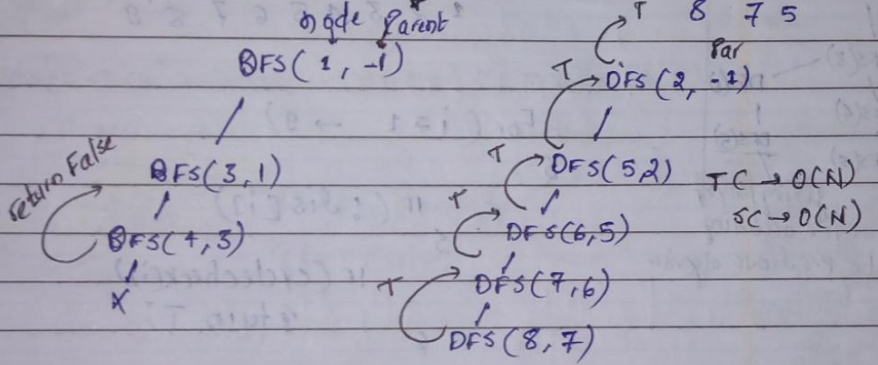




adj list

1 3
2 5
3 1 4
4 3
5 2 6 8
6 5 7
7 6 8
8 7 5

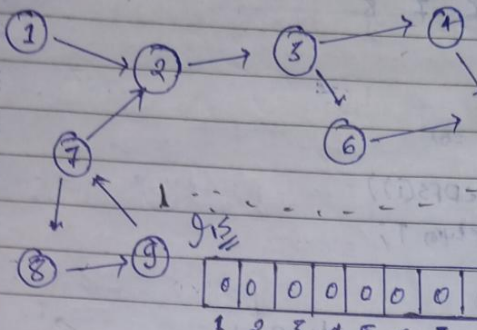
```
For( i = 1 → 8 )
{
    IF ( vis[i] == 0)
    {   IF ( cycleDFS(i))
            return T;
    }
}
```




```
bool checkForCycle ( int node, int Parent, .. )
{
    vis[node] = 1;
    For ( auto it : adj[node]) {
        IF ( visit[it] == 0) {
        IF ( checkForCycle(it, node, vis, adj))
            return true;
        }
        else IF ( it != Parent)
        { return true;
        }
    }
    return False;
}
```

bool Q. Detect cycle in Directed Graph (DFS)

{



Prev. alg. will not work here

DFS(1)
DFS(2)
DFS(3)
DFS(4)
DFS(5)
DFS(2)
DFS(5)

giving / saying cycle according to previous algorithm

vis

0	0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8	9

adjacent list

1 → 2
2 → 3
3 → 4,6
4 → 5
5 →
6 → 5
7 → 2,8
8 → 9
9 → 7

For (i = 1 → 9)

{

IF (! vis[i])

{

IF (cyclecheck(i))
return T;

}

}

vis

1	1	1	1	1	1	0	0	0
1	2	3	4	5	6	7	8	9

curr_dfs vis

1	1	1	1	1	1	0	0	0
1	2	3	4	5	6	7	8	9

F(1)
F(2)
F(5)
F(4)
F(6)
F(5)

while return work visited

Q. Cycl

1

F(7)
F(8)
F(9)
T

bool

see both cases

Don't forget

Q. To

5
2
5
4

Graph
(Dire

(DFS)

adj list
1 → 2
2 → 3
3 → 4, 6
4 → 5
5 →
6 → 5
7 → 2, 8
8 → 9
9 → 7

```
bool checkcycle(int node, vector<int> adj[], int vis[],
                int dfsvis[]) {
    vis[node] = 1;
    dfsvis[node] = 1;
    for(auto it : adj[node]) {
        if(!vis[it]) {
            if(cycdecheck(it, adj, vis, dfsvis))
                return true;
        }
        else if(dfsvis[it]) {
            return true;
        }
    }
    dfsvis[node] = 0;
    return false;
}
```

see both cases

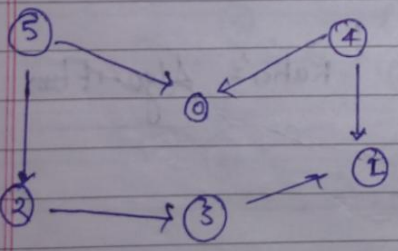
Don't forget

make this zero during ret. of current dfs

Q. Topological sorting

linear indexing of vertices such that there is an edge u → v, u appears before v in that ordering

u is before v for any pair in this order



5 4 2 3 1 0

4 5 2 3 1 0

0 -
1 -
2 - 3
3 - 1
4 - 0, 1
5 - 0, 2

Graph should be DAG (Directed acyclic graph)

```
For(i=0→5)
{
  IF(!vis[i])
  {
    dfs(i);
  }
}
```

vis: 1 1 1 1 1 1 | 1 0 0 0 0 0 | 0 1 2 3 4 5

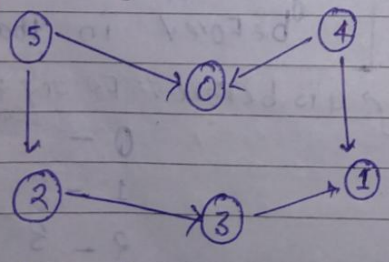
Recursive tree: ~~dfs(0)~~ → ~~dfs(1)~~ → dfs(2) → dfs(3) → dfs(4)

initiation → 4 comes before 4

stack (top): 2, 3, 1, 0

```
void FindTopoSort(int node, vector<int>& vis, stack<int>& st,
                  vector<int> adj[])
{
  vis[node] = 1;
  for(auto it : adj[node])
  {
    if(!vis[it])
    {
      FindTopoSor(it, vist, st, adj);
    }
  }
  st.push(node);
}
```

Topological Sort (BFS) | Kahn's Algorithm



indegree: 2 2 1 1 0 0 | 0 0 0 0 0 0 | 0 1 2 3 4 5

queue (top): 2, 0, 5, 4

* First push node having indegree 0 in queue data structure

Now simply implement BFS algorithm

going to 0 & 1 & reduce indegree by 1

First Push node in result ← node = 4

↳ 0, 1 → Now check if it is zero

2	2	1	1	0	0
0	1	2	3	4	5

node = 5

1	1	0	1	0	0
0	1	2	3	4	5

↳ (3), 2

↳ Yes zero Put in q.

4 5 0 2 3 1

node = 0

node = 2 → (5) ... Put in q

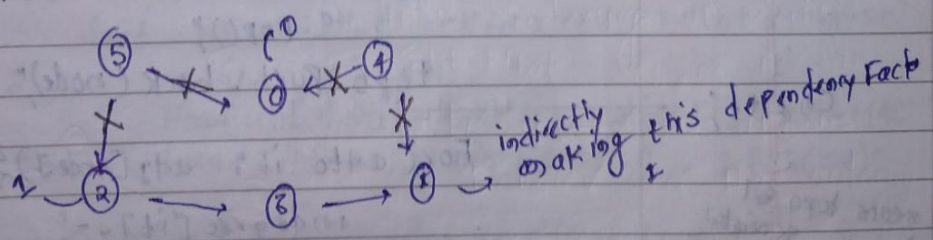
node = 3

↳ 1 ... Put in q

node = 1

intitution

↳ u → v (u should appear before v)



```
vector<int> toposort(int N, vector<int> adj[]){
    queue<int> q;
    vector<int> indegree(N,0);

    for(int i=0; i<N; i++)
    {
        for(auto it: adj[i]){
            indegree[it]++;
        }
    }

    for(int i=0; i<N; i++){
        if(indegree[i]==0){
            q.push(i);
        }
    }
    vector<int> topo;
    while(!q.empty()){
        int node = q.front();
        q.pop();
        topo.push_back(node);
        for(auto it: adj[node]){
            indegree[it]--;
            if(indegree[it]==0){
                q.push(it);
            }
        }
    }
    return topo;
}
```

For next Q^n

```
int cnt = 0;
cnt++;
```

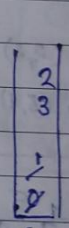
means topo sort generated

```
if(cnt==N)
    return false;
return true;
```

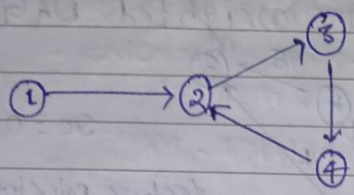
Q. Cycle

Kahn's

Q. Shortest

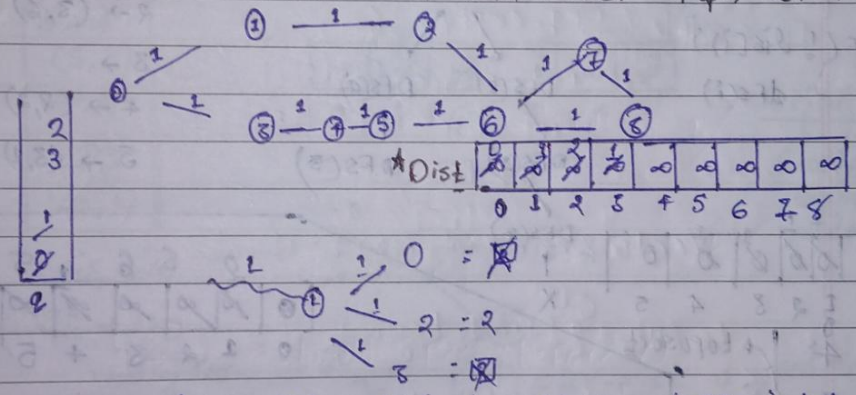


Q. Cycle check in directed Graph using BFS (Kahn's algo)



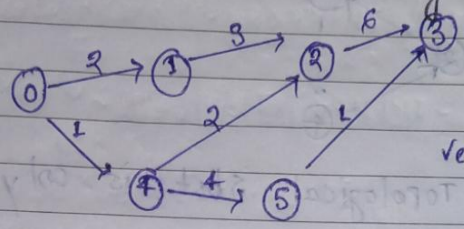
Kahn's Algo → Topological sort is only possible for DAG

Q. Shortest Distance in undirected Graph with unit weight




```
void BFS (vector<int> adj[], int N, int src)
{
    int dist[N];
    For(int i=0; i<N; i++) dist[i] = INT_MAX;
    queue<int> q; dist[src] = 0; q.push(src);

    while(!q.empty)
    {
        int node = q.front(); q.pop();

        For(auto it : adj[node]) {
            if(dist[node]+1 < dist[it]){
                dist[it] = dist[node]+1;
                q.push(it);
            }
        }
    }
    For(int i=0; i<N; i++) cout<<dist[i]<<" ";
}
```

Shortest path on a weighted DAG



src = 0

vector < pair < int, int >> adj[]

0 → (1,2) (4,1)

1 → (2,3)

2 → (3,6)

3 →

4 → (2,2) (5,4)

5 → (3,1)

For (i : 0 → 5)

{

if (! vis[i])

dfs(i)

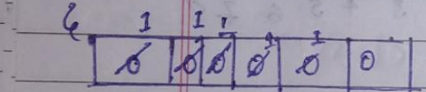
DFS(0)

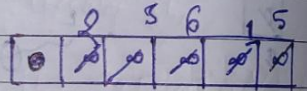
DFS(1) DFS(4)

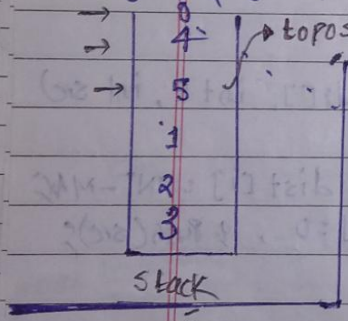
DFS(2) DFS(5)

DFS(3)

X





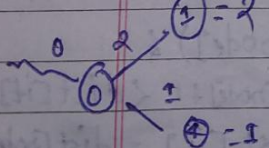


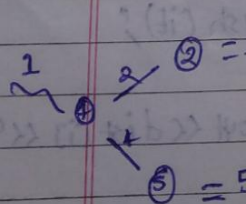
node = 0

IF(dist[node] != ∞)

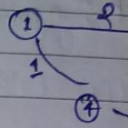
Nnn code like predict

Q1

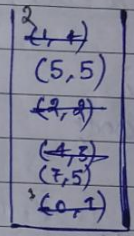




Dijkstra's



src = 1



(dis, node)

PQ → Min

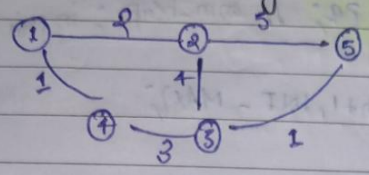
↓

having less distance will be at top

TC → O(N+E)

SC → O(N) FO

Dijkstra's Algorithm



adj list

1 → (2,2), (4,1)
2 → (1,2), (5,5), (3,4)
3 → (2,4), (4,3), (5,1)
4 → (1,1), (3,3)
5 → (2,5), (3,1)

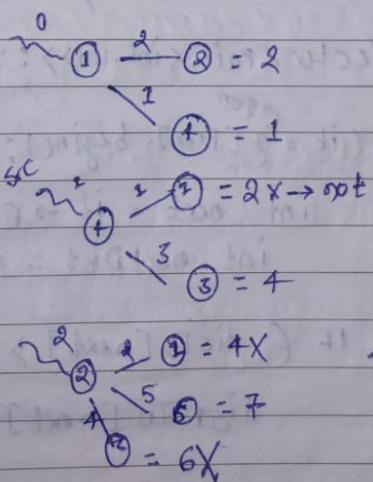
src = 1

0	1	2	3	4	5
~~∞~~	~~∞~~	~~∞~~	~~∞~~	~~∞~~	~~∞~~

(dis, node)

PQ → Min
↓
having less distance will be at top

Initialization
↓
Greedy (we are taking minimum distance at first)



= 2

= 1

= 2X → not a good path (not used)

= 4

= 4X

= 7

= 6X

= 12X

= 6X

= 8X

= 7X

= 5 ✓

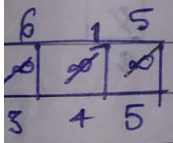
= 10 X

= 6 X

For PQ ↑

TC → O(N+E) log N

SC → O(N) + O(N)



G

0

int, int >> adj[];

0 → (1,2) (4,1)

1 → (2,3)

2 → (3,6)

3 →

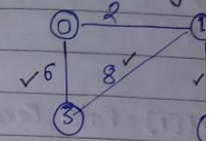
4 → (2,2) (5,4)

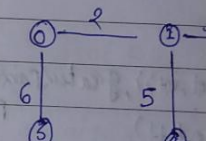
5 → (3,1)

previous

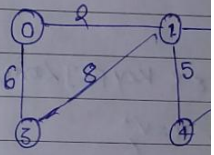
```
priority_queue < pair <int, int>, vector <pair<int,int>>,
    greater <pair <int, int>>> pq; // min_heap ; in pair => (dist)

vector < int> distTo (n+1, INT_MAX);

distTo [source] = 0;

pq.push (make_pair (0, source)); // (dist, From)

while (!pq.empty()) {
    int dist = pq.top().first;
    int prev = pq.top().second;
    pq.pop();

vector <pair <int, int>> :: iterator it;

for (it = g[prev].begin(); it != g[prev].end(); it++)
{
    int next = it -> first;
    int nextDist = it -> second;

if ( distTo [next] > distTo [prev] + nextDist)

    distTo [next] = distTo [prev] + nextDist

    pq.push( make_pair ( distTo [next], next))
}
}
```

Prim's Algo



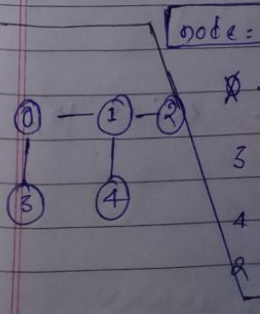


N = 5



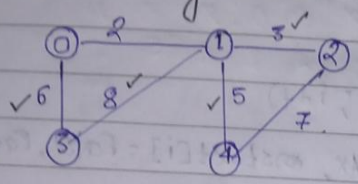
selected min

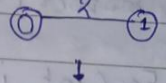
node = 0



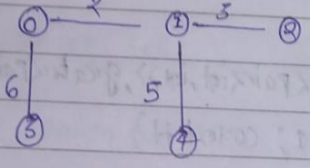
Relise on interview bit
common table island

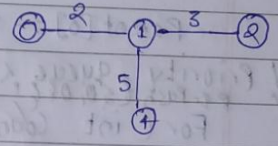
Prim's Algorithm



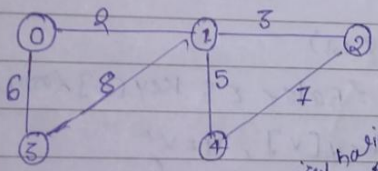


Now check all adjacent node connected to all these node (select minimal)





N = 5 (Now stop)



selected as key having minm key value & marked as false

node = 0

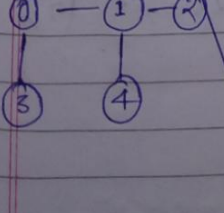
Now check all its adjacent node

	0	1	2	3	4
Key	0	2	∞	6	5
MST	T	F	F	F	F
Parent	-1	0	1	0	1

node = 2

node = 4

node = 3



node = 1 → now mark True

already in MST

3 → (already Figure 6)

4 → 5

→ 3

(Make MST)

	0	1	2	3	4
	-1	1	1	1	3

Now

0 (1 2 3 4) ← Now iterate From 1-4

Brute - Force

```
int Parent[N];
int Key[N];
bool mstset[N];

For(int i=0; i<N; i++)
    Key[i] = INT_MAX, mstSet[i] = False, Parent[i]

Key[0] = 0;
Parent[0] = -1;
// Priority_queue <Pair<int,int>, vector<Pair<int,int>>, greater<Pair<int,int>> pq;
// pq.push({0,0});
For(int count=0; count<N-1; count++)
{
    int mini = INT_MAX, u;

    int u = pq.top().second;        |  For(int v=0; v<N; v++)
    pq.pop()                        |      if(mstSet[v]==False && Key[v]<mini)
                                    |          mini = Key[v], u = v;
    mstset[u] = true;               |  mstSet[u] = true;

                                    For(auto it: adj[u]){
                                        int v = it.first;
                                        int weight = it.second;
    pq.push({Key[v], v});           |  if(mstset[v]==false && weight<Key[v])
    TC → O(log n)                   |      Parent[v] = u, Key[v] = weight;
                                    }
}

For(int i=1; i<N; i++)
    cout << Parent[i] << " - " << i << "\n";

return 0;
}
```

TC → $O(N^2)$

Disjoint set → union by Rank & Path Compression

→ Find Par()

→ Union()

every one is parent of itself

1 2 3

4 5 6 7

Parent

union(1,2) → 1 2

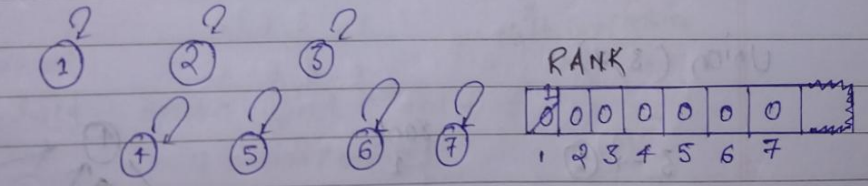
union(2,3) → 1 2 3

union(4,5) → 4 5

union(6,7) → 6 7

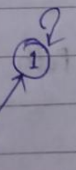
union(5,6) → 4, 5, 6, 7

union(3,7) → 1 2 3 4 5 6 7



union(1,2)

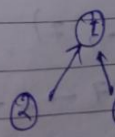
	Par	rank
1	1	0
2	2	0



Union(2,3)

	Par	rank
2	1	1
3	3	0

connect lesser rank elements to higher



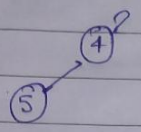
Here doesn't increase rank (as height not increase) increase rank only IF they have similar rank in that case height got increase

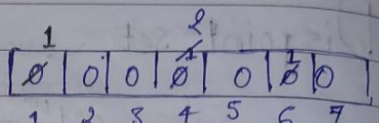
Union (4, 5)

Par rank

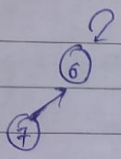
4 → 4 0

5 → 5 0



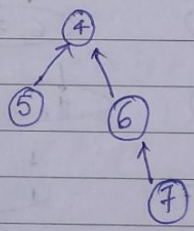


Union (6, 7)



Union (5, 6)

	Par	rank
5 →	4	1
6 →	6	1

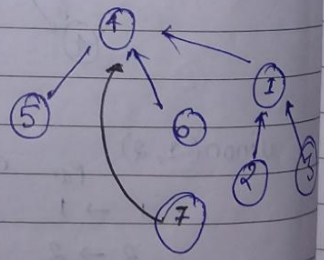


Union (3, 7)

3 → (1) rank 1

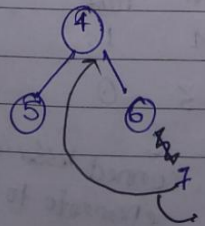
7 → 6 → (4) 2

here we do Path compression



Union (-, 7)

7 → 6 → 4



Path compression

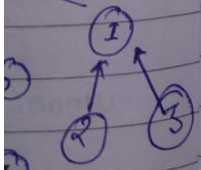
TC → O(4α) = O(4)

```
int Parent [1000000];
int rank [1000000];

void makeset() {
    For (int i=1; i<=n; i++) {
        Parent [i] = i;
        rank [i] = 0;
    }
}

int FindPar (int node) {
    IF (node == Parent [node]) {
        return node;
    }
    return FindPar ( Parent [node] );
    OR
    return Parent [node] = FindPar ( Parent [node] );
}
```

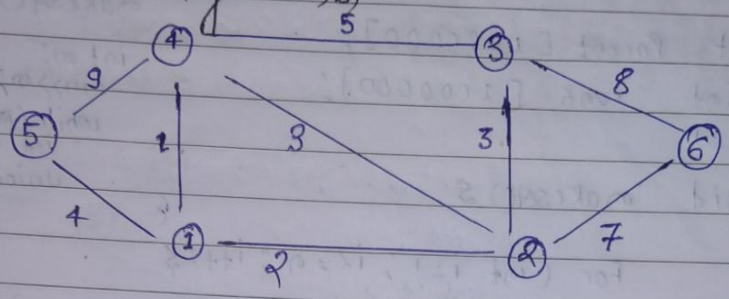
Path compression

```
void union ( int u, int v) {
    u = FindPar [u];
    v = FindPar [v];
    IF ( rank [u] < ran [v] ) {
        Parent [u] = v;
    }
    else IF ( rank [v] < rank [u] ) {
        Parent [v] = u;
    }
    else {
        Parent [v] = u;
        rank [u] ++;
    }
}
```

```
void main() {
    makeset();
    int m;
    cin >> m;
    while (m--) {
        int u, v;
        union (u, v);
    }
}
```



Kruskal's Algorithm



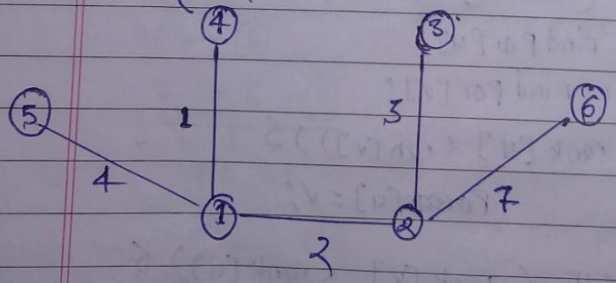
(1) Sort all edges w.r.t wt

wt	u	v
(1,	1,	4)
(2,	1,	2)
(3,	2,	3)
(5,	2,	4) ✗
(4,	1,	5)
(5,	3,	4) ✗
(7,	2,	6)
(8,	3,	6) ✗
(9,	4,	5) ✗

TC → O(M log M) + O(M)
↓ To sorted edges ↓ to traverse all edges

SC → O(M)

First check belong to different compo.



```
struct node {
    int u;
    int v;
    int wt;
node (int first, int second, int weight) {
        u = first;
        v = second;
        wt = weight;
    }
};

bool comp (node a, node b) {
    return a.wt < b.wt;
}

int FindPar (int u, vector<int> &parent) {
    if (u == parent[u]) return u;
    return FindPar (parent[u], parent);
}

void Union (int u, int v, vector<int> &parent, vector<int> &rank) {
    u = FindPar (u, parent);
    v = FindPar (v, parent);

    if (rank[u] < rank[v]) {
        parent[u] = v;
    }
    else if (rank[v] < rank[u]) {
        parent[v] = u;
    } else {
        parent[v] = u;
        rank[u] ++;
    }
}
```

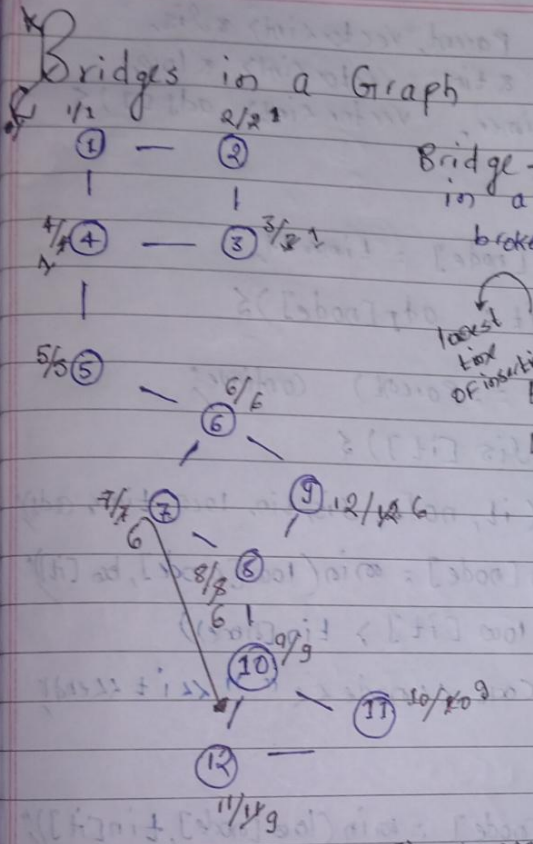
O(M log M) + O(M)
↓
to traverse all edges

red gs

```
int main(){
    int N, m;
    cin >> N >> m;
    vector <node> edges;
    For (int i=0; i<m; i++){
        int u, v, wt;
        cin >> u >> v >> wt;
        edges.push_back (node(u, v, wt));
    }
    sort (edges.begin(), edges.end(), comp);
    vector <int> Parent (N);
    For (int i=0; i<N; i++)
        Parent[i] = i;
    vector <int> rank (N, 0);
    int cost = 0;
    vector < pair <int, int>> mst;
    For (auto it: edges){
        if (findPar (it.v, Parent) != findPar(it.
            cost = it.wt;
            mst.push_back ({it.u, it.v});
            Union (it.u, it.v, Parent, rank);
        }
    }
    cout << cost << endl;
    For (auto it: mst) cout << it.first << " - " << it.sec
    return 0;
}
```

** low → means i can lowest insertion time among all its adjacents = (8-10) (5-6) (4-5)

Bridges in a Graph



Bridge - Those edges are called bridge in a graph who removal graph broken in two or more components

lowest time of insertion

low [it] > tin [node] → bridge

↳ low of adjact greater than time of insertion of node

low [ ] of adjacent

low [it] > tin [node]

9 8 means is another way to reach the node

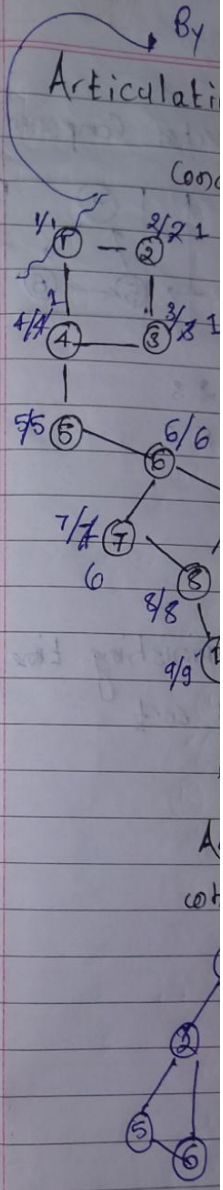
* we are only updating value low whenever dfs for adjacent node is completed

then checking

TC → O(N+E), SC → O(2N)

```
int main()
{
    vector<int> tin(n,-1);
    vector<int> low(n,-1);
    vector<int> vis(n,0);
    int time=0;
    for(int i=0;i<n;i++){
        if(!vis[i])
            dfs(i,-1,vis,tin,low,timer,adj);
    }
}
```

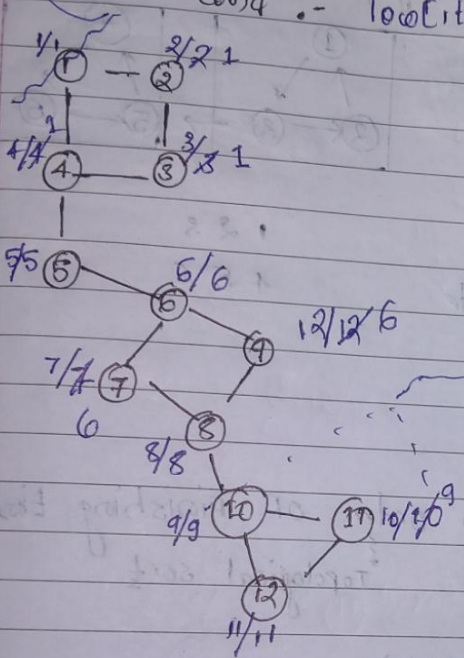
```
void dfs (int node, int parent, vector<int> &vis,
          vector<int> &tin, vector<int> &low,
          int &timer, vector<int> adj[]) {
    vis[node] = 1;
    tin[node] = low[node] = timer++;
    for (auto it : adj[node]) {
        if (it == parent) continue;
        if (!vis[it]) {
            dfs(it, node, vis, tin, low, timer, adj);
            low[node] = min(low[node], low[it]);
            if (low[it] > tin[node])
            {
                cout << node << " " << it << endl;
            }
        }
        else
        {
            low[node] = min(low[node], tin[it]);
        }
    }
}
```



By removing 1 there is no connection

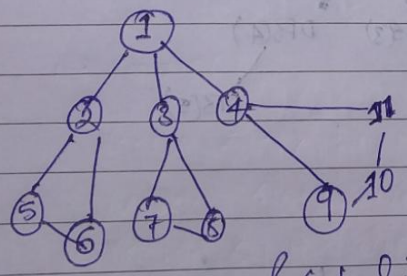
Articulation point

Cond^n :- low[it] >= tin[node] && Par != -1



9 >= 9 → 10 can't reach 11 before 10, so it is articulation point

Another cond^n:
when 1 will be articulation point



⇒ if(child > 1 && Par != -1)

if(!dis)
{

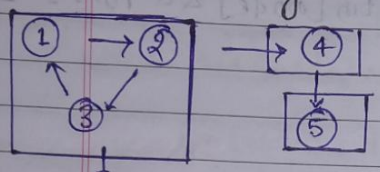
Outside of for loop

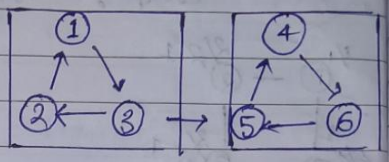
child++

}

→ IF(Parent == -1 && child > 1) { isArticulation[node] = 1; }

Kosaraju's Algorithm (strongly Connected Component)



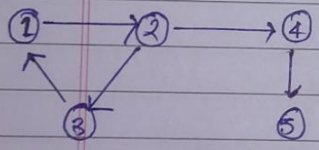


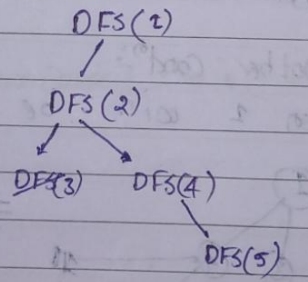
From every node we can reach every other node

1 2 3
4
5

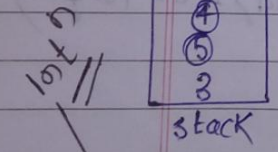
1 2 3
4 5 6

① Sort all nodes in order of Finishing time
↳ Topological sort





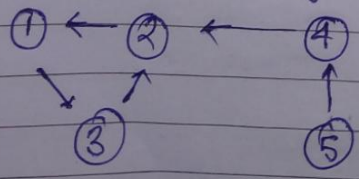
Q.



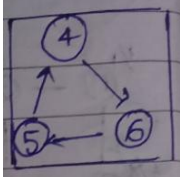
int a

1

② Transpose the graph



Component)



shing time
rt

③ DFS according to Finishing - time

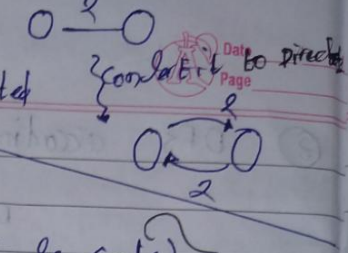
DFS(1)
DFS(3) → First SCC
DFS(2)

DFS(4)
DFS(5)

```
stack<int> st;
vector<int> vis(n,0);
for(int i=0; i<n; i++){
    if(!vis[i]){
        dfs(i, st, vis, adj);
    }
}
vector<int> transpose[n];

for(int i=0; i<n; i++){
    vis[i]=0;
    for(auto it: adj[i]){
        transpose[it].push_back(i);
    }
}
while(!st.empty()){
    int node = st.top();
    st.pop();
    if(!vis[node]){
        cout<<"SCC:";
        revDFS(node, vis, transpose);
        cout<<endl;
    }
}
```

Bellman Ford Algorithm

in order to implement Bellman Ford to undirected graph

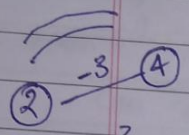


condition to directed

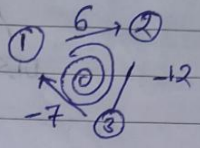
↳ DG (-ve cycle) only

+ve
-ve

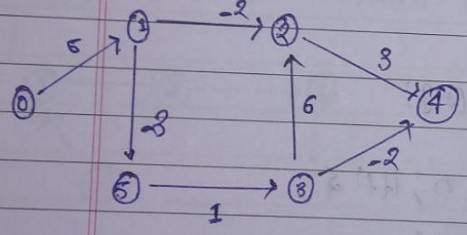
Dijkstra's Algo



will get trapped in infinite loop



not work in -ve cycle (But will detect negative cycle)



src = 0

u ✓	wt
(3, 2)	6
(5, 3)	1
(0, 1)	5
(1, 5)	-3
(1, 2)	-2
(3, 4)	-2
(2, 4)	3

(•) (Relax all the edges N-1 times) ← N-1 times

Q.

```
if ( dist[u] + wt < dist[v] )
    dist[v] = dist[u] + wt
```

dist[] →

0	∞	∞	∞	∞	∞
0	1	2	3	4	5

5 3 -1 -3 -2

⇒

0	5	3	3	1	2

× dist[3] + 6 < dist[2]

× dist[5] + 1 < dist[3]

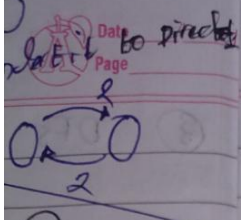
dist[0] + 5 < dist[1]

dist[1] + -3 < dist[5]

dist[1] + -2 < dist[2]

dis[3] + -2 < dist[4]

dist[2] + 3 < dist[4]



only not work in -ve cycle (But will detect negative cycle)

✓	cost
2)	6
3)	1
1)	5
5)	-3
2)	-2
4)	-2
4)	3

⇒ [0|5|3|3|1|2]

how to detect negative cost cycle

↓

Try to relax one more time, if dist gets reduced means there is -ve cost cycle

```
int inF = 10000000;
vector<int> dist (N, inf);
dist [src] = 0;

For (int i=1; i<=N-1; i++) {
        For (auto it: edges) {
            IF( dist [it.u] + it.cost < dist [it.v]) {
            dist [it.v] = dist [it.u] + it.cost;
        }
    }
}
int Fl = 0;
For (auto it: edges) {
        if ( dist[it.u] + it.cost < dist[it.v] ) {
            cout << "Negative cycle";
            Fl = 1;
            break;
        }
}
if (Fl) {
    For (int i=0; i<N; i++) {
        cout << i << " " << dist[i] << endl;
    }
}
```

Q. Rotten Orange

→

	0	1	2	3
0	⊗			⊗
1	⊗			0
2	0	0		0
3				0

4×4

BFS

(1,0)
(~~0,1~~)
(0,0)

Q
(i, j)